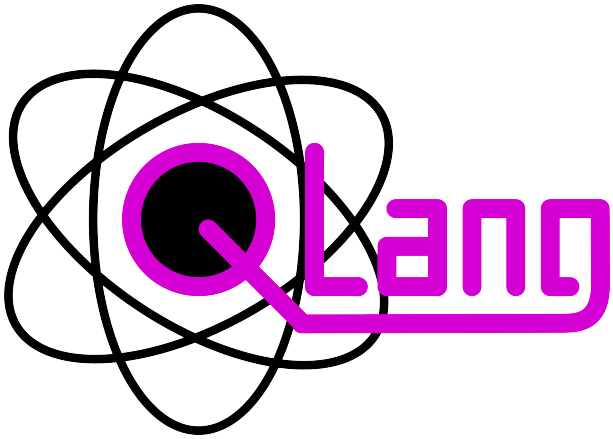

QLang Programming Language

Language Specification & Documentation

Created by:

Kamil Widzyk
Maciej Świątek
Jan Wida

Contents

1 Introduction

QLang is a high-level programming language designed for quantum computing and distributed quantum networking. It provides a simple and intuitive syntax for expressing quantum algorithms, operations, and communication between separate quantum nodes called places, making it accessible to both beginners and experienced programmers in the field of quantum computing.

1.1 Overview

A QLang program is made up of top-level items: `place` declarations, function declarations, import directives, and statements. A place is QLang's primary unit of concurrent execution. Each place has its own scope, defines its own functions, and runs as a separate process. Code written outside any named place belongs to the implicit `global` place, which only executes if it contains at least one statement or function declaration.

The language supports five built-in variable types: `state`, `obs`, `num`, `text`, and `any`. Quantum-specific operations, including the `measure` keyword and all built-in gates, are first-class constructs in QLang, not library calls. The language natively supports quantum gates such as `superpose`, `shift`, `not`, `dual_not`, `phase_not`, `entangle`, `entangle_phase`, and `swap`. QLang is interpreted. Source files with the extension `.ql` are parsed by an ANTLR4-generated parser and executed by the QLang interpreter written in Python.

1.2 Quick Example

The following complete QLang program creates a Bell state (a maximally entangled two-qubit state), measures both qubits, and prints the results:

```
place Alice {
    state a, b;

    superpose a;           // Put qubit 'a' into superposition
    entangle a -> b;       // Entangle 'a' with 'b' to create a Bell state

    obs ma = measure a;
    obs mb = measure b;

    println("Measurement A: %d" % ma);
    println("Measurement B: %d" % mb);
}
```

After running, both measurements will always agree, both `0` or both `1`, which is the defining property of a Bell state.

1.3 Design Principles

1.3.1 Classical and Quantum Code in One Language

QLang does not separate quantum and classical concerns into different APIs or files. A variable of type `obs` and a variable of type `state` (qubit) can appear side-by-side in the same scope. Measurement, when invoked on a `state`, collapses the qubit and returns an `obs` value indicating the outcome: `F` (false) if the qubit collapsed to $|0\rangle$, and `T` (true) if it collapsed to $|1\rangle$.

1.3.2 Structured Error Reporting

When the interpreter encounters an error, whether a syntax mistake, an undefined variable, or an illegal runtime operation, it displays a detailed error box that highlights the exact source location, shows surrounding context lines, and explains the cause.

1.3.3 Modular Structure via Places

QLang organizes code into units called `places`. Each `place` has its own name, its own variables, and runs independently from other places. Places can communicate with each other using the built-in `send` and `receive` instructions, which transfer both classical and quantum data. This makes it natural to write distributed programs, such as quantum communication protocols, directly in QLang.

2 Installation & Setup

2.1 Requirements

Before you can run a QLang program, ensure your system meets the following requirements:

- **OS: Windows**
- **Python 3.14**.
- **uv**.
- **Java Runtime Environment (JRE)**.

2.2 Running a QLang Program

QLang source files use the `.ql` extension. To execute a QLang program, pass the file path to the interpreter.

```
run.bat <path-to-.ql-file>
```

2.3 Project Structure

The QLang repository is organized as follows:

- `int/` — Contains the core interpreter logic, AST generation, memory management, and runtime contexts.
- `antlr/` — Contains the ANTLR4 grammar file (`QLang.g4`) and tools for generating the parser.
- `examples/` — A collection of sample `.ql` scripts demonstrating language features.
- `docs/` — The LaTeX source for this documentation.
- `run.bat` — The script to generate the parser and execute `.ql` files.

3 Language Basics

This chapter covers the fundamental building blocks of QLang: how programs are structured, the available data types, variables, operators, and control flow constructs.

3.1 Program Structure

A QLang source file (extension `.ql`) consists of a sequence of top-level items. Each item is one of the following:

- A `place` declaration.
- A function declaration (at global place).
- A statement (at global place).

All named places start executing simultaneously when the program is launched. The implicit global place is only executed if it contains at least one statement or function declaration.

3.2 Data Types

QLang has five built-in variable types, designed to cover both classical and quantum computation:

Type	Description
`state`	A quantum state (qubit or qubit register). Can be in superposition and entangled with other qubits. The stored quantum information is only accessible via the `measure` keyword; states cannot be read, assigned, or printed directly.
`obs`	A classical observation. A scalar `obs` stores a boolean value; a sized array `obs x[N]` stores an N-bit array and allows access to individual bits by index. Used for branching, control, and classical post-processing after measurement.
`num`	A universal numeric type. It dynamically handles both integers and floating-point numbers, automatically adapting its internal representation based on the assigned value and performed operations. Used for all classical numerical computations.
`text`	A string of characters. Used for textual data, print formatting, and string operations. Supports the same indexing modes as arrays.
`any`	A generic type usable in function parameter declarations and variable declarations. A parameter of type `any` accepts a value of any type and infers the actual type at the call site.

3.2.1 Type Casting

QLang provides the built-in `num()` function to cast and parse values into the `num` type. It can parse integer, floating-point, hexadecimal (prefixed with `0x`), and binary (prefixed with `0b`) numbers from text strings, automatically ignoring surrounding whitespace. If the string is not a valid number, the interpreter will raise a runtime error.

```
text hex_str = "0x1A";
text bin_str = "0b1010";
text float_str = " 3.1415 ";

num h = num(hex_str);    // 26
num b = num(bin_str);    // 10
num f = num(float_str);  // 3.1415
```

3.3 Variables

Variables are declared by specifying a type, a name, optional size specifiers and an optional initial value.

3.3.1 Declaration Syntax

```
num pi = 3.14159;            // floating-point number
num count = 10;              // num accepts integer values too
obs result[16];              // 16-bit observation array (all bits default to 0)
obs counter[8] = 0 >8> obs;  // Convert integer 0 to an 8-bit observation array
text greeting = "Hello!";    // string
```

3.3.2 Arrays

Any variable type can be declared as an array by appending one or more size specifiers in square brackets. Arrays are zero-indexed. A size can be a fixed integer literal or an expression evaluated at runtime. Using `[?]` as the size declares a dynamic array, whose length adjusts automatically when elements are appended using the `+=` operator:

```
num matrix[4][4];            // two-dimensional 4x4 array

// Size inferred automatically from the initialiser
num a[?] = [1, 2, 3, 4, 5]; // size is 5, inferred from the list
println(#a);                 // 5

// Runtime-computed size
num n = 8;
num dynamic[n];              // size determined at runtime
```

3.3.3 Constants

The `const` keyword can be applied to any variable declaration to prevent reassignment after initialisation. A variable can also be declared mutable first and then locked with `const x;`, once frozen, further assignments raise a runtime error. Variable of type `state` cannot be declared as constants or frozen. Attempting to apply `const` to a `state` variable will raise a runtime error.

```
const obs MAX_SIZE[8] = 100;   // sized obs constant
const num EPSILON = 0.0001;    // num constant


num x = 10;
x++;       // still mutable
const x;   // freeze: further assignments raise an error
```

3.4 Operators

3.4.1 Arithmetic Operators

Operator	Description
+	Addition
−	Subtraction
*	Multiplication
/	Division
%	Modulo
**	Exponentiation

3.4.2 Unary Operators

Operator	Description
+x	Unary plus, returns the value unchanged.
−x	Unary minus, negates the value. If applied to a boolean type, behaves as logical NOT and returns the negated boolean value (T or F).
! x	Logical NOT. If x is a boolean type, returns the negated boolean (T or F); otherwise, returns 1 if x is zero, 0 otherwise.

3.4.3 Comparison and Logical Operators

Operator	Description
==	Equal to
! =	Not equal to
<, >, <=, >=	Relational comparisons
&&	Logical AND
| |	Logical OR

3.4.4 Increment and Decrement

```
num i = 0;
i++;     // post-increment
++i;     // pre-increment
i--;     // post-decrement
--i;     // pre-decrement
```

3.4.5 Compound Assignment Operators

In addition to plain assignment (`=`), QLang supports compound operators: `+=`, `-=`, `*=`, `/=`, `%=`, `**=`. The operators `&=` and `|=` are logical compound assignments equivalent to `x = x && y` and `x = x || y` respectively.

All compound assignment operators return the new value of the variable, so they can be used inside expressions.

3.4.6 Assignment Expression

Plain assignment (`=`) is also an expression and returns the assigned value. Assignment must always target a variable; attempting to assign to a non-variable expression (e.g. `5 = x`) raises a runtime error.

3.4.7 The # Size Getter

The `#` prefix operator returns the current length of an array or a `text` string. For dynamic arrays, this reflects the actual number of elements after any appends:

```
num items[?] = [10, 20, 30, 40, 50];
println(#items);    // 5
text message = "hello";
println(#message);  // 5
```

3.4.8 The `reset` Operator

The `reset` operator restores a variable to its initial value as declared, or to the type default if no initial value was given. It works on scalars, strings, and multi-dimensional arrays. It returns the value after reset.

```
// reset scalar
num x = 5;
x = 99;
reset x;
println(x);  // 5

// reset text
text s = "hello";
s = "world";
reset s;
println(s);  // hello

// reset array
num arr[2] = [1, 2];
arr[0] = 99;
reset arr;
println(arr[0]);  // 1
```

3.4.9 String Formatting Operator

The `%` operator, when the left operand is a `text` string, performs printf-style interpolation:

```
num x = 42;
num y = 3.14159;
println("The answer is %d" % x);
println("x = %d, y = %.2f" % [x, y]);  // list of values

text fmt = "Hello, %s!";
println(fmt % "world");
```

3.4.10 The Type Operator $

The $ prefix operator dynamically evaluates and returns the type name of an expression as a text string (e.g., "num", "text"). This is especially useful for type-checking when working with any parameters.

```
num n = 42;
text t = "hello";
num arr[?] = [1, 2, 3];

println($n);   // num
println($t);   // text
println($arr); // list
```

3.4.11 Bitwise Stream Conversions

QLang provides special stream operators for converting data between integer numbers and arrays of binary bits. These operators are strictly typed and will raise an error if used with unsupported targets.

- `array >> num` — Evaluates an array of binary values (0 or 1) and converts it into a single `num` integer. The first element of the array is treated as the Least Significant Bit (LSB).
- `expr >size> obs` — Takes a numeric expression and extracts a specified `size` number of bits, converting it into an array of observations (`obs`). The least significant bit becomes the first element of the resulting array.

```
// Convert an array of bits to an integer
// The first element is the Least Significant Bit (LSB)
num bits[?] = [1, 0, 1];     // 1*2^0 + 0*2^1 + 1*2^2 = 1 + 4 = 5
num val = bits >> num;       // val is 5

// Convert an integer to an array of observation bits
// Extracts the given number of bits (4 in this case)
obs out[4] = 5 >4> obs;    // out is [1, 0, 1, 0]
```

3.4.12 The Parent Operator ^

The ^ operator explicitly accesses a variable in an outer scope. Multiple carets chain upward, one level per caret. See Section **??** for a full description.

3.5 Control Flow

3.5.1 Conditional Statements

QLang supports standard `if` / `else if` / `else` branching. `elif` is a shorthand alias for `else if`. When the branch body is a single statement, curly braces may be omitted. An inline ternary form `cond ? a : b` is also available as an expression:

```
num x = 75;

if (x > 100) {
    println("Large");
} elif (x > 50) {
    println("Medium");
} else {
    println("Small");
}

// if without braces
if (x > 0) println("Positive");

// Ternary operator
text label = x > 50 ? "big" : "small";
println(label);
```

3.5.2 For Loop

The `for` loop iterates over a numeric range with an optional step. The syntax is `for <var> from <start> to <end> step <step>`. The range is exclusive. The `start`, `end`, and `step` boundaries can be dynamically evaluated expressions, meaning they are re-evaluated in every iteration. The loop variable is created automatically and is only accessible inside the loop body. When `step` is omitted, the interpreter infers it automatically: `+1` if `end > start`, `-1` otherwise.

```
// ascending for loop
for i from 1 to 5 {
    println(i);  // 1 2 3 4
}

// descending for loop
for i from 5 to 1 {
    println(i);  // 5 4 3 2
}

// custom step for loop
for i from 0 to 10 step 2 {
    println(i);  // 0 2 4 6 8
}
```

3.5.3 While Loop

The `while` loop repeats its body as long as the condition evaluates to `true` for boolean expressions, or is greater than zero for numeric expressions.

```
// while loop
num x = 0;
while (x < 5) {
    println(x);
    x++;
}
// 0 1 2 3 4
```

3.5.4 Iterate Loop

The `iterate` loop traverses every element of an array. The current element is bound to a named variable. An optional `index` clause provides a second variable that holds the current zero-based index. Both `break` and `continue` work inside `iterate`:

```
// iterate over array
num arr[4] = [10, 20, 30, 40];
iterate arr as val {
    println(val);  // 10 20 30 40
}

// iterate with index
iterate arr as val index i {
    println("%d: %d" % [i, val]);
    // 0: 10
    // 1: 20
    // 2: 30
    // 3: 40
}

// break and continue work inside iterate
iterate arr as val {
    if (val == 20) continue;
    if (val == 40) break;
    println(val);  // 10 30
}
```

3.5.5 Break and Continue

`break` exits the nearest enclosing loop immediately. `continue` skips the rest of the current iteration and proceeds to the next one.

3.6 Functions

Functions are declared with the `function` keyword. Parameters are typed, can have default values, and may be arrays. Supported parameter types are `num`, `obs`, `state`, `text`, and `any`. A parameter of type `any` accepts a value of any type and infers the actual type at the call site. A function returns a value with the `return` statement.

```
function add(num a, num b) {
    return a + b;
}

function greet(text name = "World") {
    println("Hello, " + name + "!");
}

function power(num base, num exp) {
    return base ** exp;
}

// any parameter accepts any type
function print_value(any x) {
    println(x);
}

greet();                            // Hello, World!
greet("Alice");                     // Hello, Alice!
println(add(3, 4));                 // 7
println(power(exp = 3, base = 2));  // 8
print_value(42);                    // 42
print_value("hello");               // hello
```

3.6.1 Named Arguments

Arguments can be passed by name, which allows them to be supplied in any order and makes call sites more self-documenting:

```
function rect(num width, num height = 1) {
    return width * height;
}

// Positional arguments
println(rect(5, 3));           // 15

// Mixed: positional + named
println(rect(5, height = 3));  // 15

// All named, any order
println(rect(height = 4, width = 6));  // 24

// Using default value
println(rect(7));              // 7
```

3.6.2 Variadic Parameters

A function can accept a variable number of arguments using the `...` prefix, for example `function sum(...values)`. Variadic arguments do not have a type written before the name and may mix values of different types. Inside the function they are available as an array:

```
function sum(...values) {
    num s = 0;
    for i from 0 to #values {
        s += values[i];
    }
    return s;
}

println(sum(1, 2, 3));         // 6
println(sum(10, 20, 30, 40));  // 100

// Regular parameters can precede the variadic one
function prefixed(num x, ...rest) {
    println("x=" + x + " count=" + #rest);
}

prefixed(5, 10, 15);   // x=5 count=2
```

3.7 Input and Output

3.7.1 Output

QLang provides three output statements:

- `print(...)` — prints values without a trailing newline.
- `println(...)` — prints values followed by a newline.
- `debug(x)` — prints detailed diagnostic information. For more details and advanced usage, see Section **??**.

```
// Basic output
print("no newline");
println(" with newline");
println();                      // blank line

// Multiple arguments
println(1, 2, 3);               // 1 2 3

// Printf-style formatting
num val = 42;
println("The answer is %d" % val);
println("x = %d, y = %.2f" % [val, 3.14]);
```

3.7.2 Input

The `input` statement reads a value from the user into a variable. An optional constraint limits the accepted input. For numeric variables the constraint specifies a value range using the notation `min..max`. For `text` variables the constraint specifies the allowed length in characters. The interpreter keeps prompting until a valid value is entered.

```
obs age[8];
input(age);                     // read any value

obs score[8];
input(score, 0..100);           // accept only values in range [0, 100]
```

3.8 Time and Delays

QLang provides two ways to pause the execution of a program block: the built-in `wait` statement and the `sleep()` function.

The `wait` statement takes a numeric expression followed by a time unit. Supported time units are: `sec`, `second`, `seconds`, `ms`, `millis`, `millisecond`, `milliseconds`, `min`, `minute`, `minutes`, `hour`, `hours`.

The built-in `sleep()` function takes a single numeric argument representing the delay in seconds.

```
// Pause execution using wait keyword
wait(2 sec);
wait(500 milliseconds);


// Pause execution using the built-in sleep function
// sleep takes a single argument in seconds
sleep(1.5);
```

4 Scope and Arrays

4.1 The Parent Operator ^ and Scope Hierarchy

4.1.1 Scope in QLang

In QLang, every block of code creates a new scope level, forming a linked chain of scopes. Both reading and writing a variable automatically walk this chain upward until the variable is found. The parent operator ^ is needed only when you want to explicitly skip the current scope and target a specific outer level - for example, when a local variable shadows an outer one.

4.1.2 How ^ Works

The parent operator ^ explicitly selects a scope one level above the current one. Multiple carets chain upward, one level per caret:

- **^x** accesses **x** in the parent scope
- **^^x** accesses **x** in the grandparent scope
- **^^^x** accesses **x** three levels up

Once the target scope is reached, the variable is looked up only in that specific scope; the automatic upward walk does not apply at that point. The operator supports all standard operations: assignment, compound assignment, increment and decrement, as well as indexed access on arrays.

The following example demonstrates reading and writing through the scope hierarchy:

```
num x = 1;
if (1) {
    num x = 2;
    if (1) {
        println("%d" % ^x);   // 2 - reads parent x
        ^x = 7;
        println("%d" % ^x);   // 7 - assigns to parent x
        (^x)++;
        println("%d" % ^x);   // 8
        ^x += 3;
        println("%d" % ^x);   // 11
        println("%d" % ^^x);  // 1 - reads grandparent x
    }
}
```

Array indexing works the same way. The parent operator applies to the variable, and the index is evaluated afterward:

```
num arr[2] = [10, 20];
if (1) {
    if (1) {
        println("%d" % ^^arr[0]);  // 10
        println("%d" % ^^arr[1]);  // 20
    }
}
```

4.2 Advanced Array Structures and Indexing

QLang supports three indexing modes for arrays: simple indexing, range indexing, and list indexing. All three resolve negative indices automatically against the container length.

4.2.1 Simple Indexing and Negative Indices

A single integer index selects one element. Negative values count from the end of the array: an index `i` where `i < 0` is resolved as `i + length` before access.

```
num arr[5] = [10, 20, 30, 40, 50];
println(arr[0]);   // 10
println(arr[-1]);  // 50
println(arr[-2]);  // 40
```

4.2.2 Range Indexing

A range index `start..end` selects a contiguous subarray. The range is left-inclusive and right-exclusive: element at `start` is included, element at `end` is not. Negative values are resolved the same way as in simple indexing.

```
num arr[5] = [10, 20, 30, 40, 50];
println(arr[1..3]);   // [20, 30]
println(arr[0..-1]);  // [10, 20, 30, 40]
```

4.2.3 List Indexing

A list index `[[a, b, c]]` selects elements at the given positions in arbitrary order. The double-bracket syntax is used for a literal list of indices. To pass an existing array variable as the index list, use a single bracket. The interpreter detects that the expression is an array and treats it as a list of indices automatically. All three indexing modes also work on `text` strings.

```
num arr[5] = [10, 20, 30, 40, 50];
println(arr[[0, 2, 4]]);   // [10, 30, 50]
println(arr[[4, 0, 2]]);   // [50, 10, 30]
num idx[3] = [0, 2, 4];
println(arr[idx]);   // [10, 30, 50]
```

4.3 Scope Lifetime

In QLang, variables are bound to the specific scope they are declared in and are destroyed as soon as that scope ends. This is particularly important when working with loops.

```
num sum = 0;
for i from 1 to 3 {
    // This variable is created fresh in every iteration
    num temp = i * 10;
    sum += temp;
}
// Trying to access 'temp' or 'i' here would result in an error
```

4.4 Scope Interaction with `const`

The `const` keyword interacts with the scope hierarchy in two distinct ways, depending on whether you are declaring a new variable or modifying an existing one.

If you declare a new constant, it creates a fresh variable in the current scope, potentially shadowing outer variables. However, if you apply `const` to an already existing variable, the interpreter searches up the scope tree to find the nearest `x` and marks that specific instance as constant.

```
num global_val = 10;
num local_val = 20;

if (T) {
    // This shadows the outer 'local_val'.
    const num local_val = 50;

    // Making an existing outer variable constant from within a local scope.
    const global_val;
}
// The outer 'local_val' remains 20 and is NOT constant.
local_val = 21; // Valid

// global_val = 11; // Not valid, cannot reassign const variable
```

4.5 Functions and Closures

Functions in QLang act as closures. When a function is declared, it captures the exact scope hierarchy present at the moment of its creation (its `closure_scope`).

When the function is later called, the interpreter creates a new execution scope for the function's body. The parent of this new scope is set to the captured closure scope, not the scope from which the function was called.

5 Quantum Computing

This chapter introduces the quantum-specific features of QLang: the built-in quantum types, the primitive quantum gates available in the language, how measurement works, and two canonical example protocols: quantum teleportation and superdense coding.

5.1 Quantum Types: `state` and `obs`

QLang exposes two first-class types for quantum programming:

- `state`: a quantum register (one or more qubits). A `state` can be put into superposition and entangled with other `state`s using the built-in gate primitives. The stored quantum information is only accessible via the `measure` keyword; states cannot be read, assigned, or printed directly.
- `obs`: a classical observation. A scalar `obs` stores a boolean value. Used for branching, control, and classical post-processing after measurement.

A variable declaration can specify arrays and bit-widths. `state q[2]` declares a two-qubit register; `obs result[8]` declares an 8-bit classical register.

5.2 Primitive Gates

QLang provides primitive quantum gates as first-class language constructs. Each gate, excluding swap, has a short uppercase name and a lowercase alias. The supported gates are listed below.

Gate	Alias	Description
`H`	`superpose`	Hadamard gate, creates superposition on a single qubit.
`S`	`shift`	Phase gate, introduces a phase shift on a single qubit.
`X`	`not`	Pauli-X, flips the computational basis state.
`Y`	`dual_not`	Pauli-Y, combines a bit flip and a phase flip.
`Z`	`phase_not`	Pauli-Z, applies a relative phase flip between $\|0\rangle$ and $\|1\rangle$.
`CNOT`	`entangle`	Controlled-NOT, entangles two qubits.
`CZ`	`entangle_phase`	Controlled-Z, applies a phase flip conditioned on the control qubit.
`-`	`swap`	Swap gate, swaps the states of two qubits.

Single-qubit gates act on one `state` variable in-place. Most multi-qubit gates (like `CNOT` or `CZ`) use the arrow syntax (`->`) to indicate the control-to-target relationship. However, the `swap` gate has its own syntax and simply takes two space-separated qubits: `swap q1 q2;`.

```
state q1;
state q2;

// Single-qubit gates
H q1;            // Apply Hadamard to q1
superpose q1;    // Alias for H q1
X q2;            // Apply Pauli-X to q2
not q2;          // Alias for X q2

// Two-qubit gates (control -> target)
CNOT q1 -> q2;       // Entangle: q1 is control, q2 is target
entangle q1 -> q2;   // Alias for CNOT q1 -> q2

// Swap gate
swap q1 q2;          // Swap the states of q1 and q2
```

5.3 Measurement

Measurement forces a `state` to collapse into a classical outcome. The `measure` keyword returns an `obs` value indicating the outcome: `F` (false) for $|0\rangle$, and `T` (true) for $|1\rangle$. Measuring an array of qubits at once produces a list of `obs` values, which can be assigned directly to an `obs` array.

QLang also provides `measureX`, which applies a Hadamard gate before measuring, effectively measuring in the X basis rather than the Z basis.

The following example puts a qubit into superposition, measures it, and prints the result:

```
state q;
H q;
obs result = measure q;
println(result >> num);
```

Measurement is nondeterministic when applied to a superposed or entangled state.

6 Places and Inter-Place Communication

This chapter describes QLang's concurrency model: how a program is divided into independently executing `place`s, how each place gets its own console, and how places exchange both classical and quantum data using the built-in `send`, `receive`, and `available` constructs.

6.1 Place Declaration

A `place` is QLang's primary unit of concurrent execution. Each `place` declaration creates an isolated context that runs in its own operating-system process. Places start executing simultaneously when the program is launched.

The syntax is:

```
place Name {
    // statements and function declarations
}
```

`Name` must consist of letters, digits, and underscores. The name `global` is reserved and cannot be used. Place names must be unique within a program. Each place has its own scope, its own variables, and its own set of functions. A `place` cannot be defined inside an imported file; it must be declared in the main source file.

The following example declares two places that run in parallel:

```
place Alice {
    println("Hello from Alice!");
}

place Bob {
    println("Hello from Bob!");
}
```

Both places start at the same time. Their output may appear in non-deterministic order.

6.1.1 The Global Place

Code written outside any `place` declaration belongs to the implicit `global` place. The global place behaves like any other place: it runs as a separate process and gets its own console window. If the source file contains no `place` declarations at all, the entire program executes inside the global place.

6.1.2 Functions Inside Places

Functions declared inside a place are local to that place. They cannot be called from another place. Each place can also declare its own variables, constants, and loops without interfering with other places.

6.2 Console and I/O per Place

Every place opens a separate console window when the program starts. The window title includes the place's name (e.g., `Place: Alice`). All `print`, `println`, and `input` calls from a place are routed to that place's console window.

6.2.1 `show_console(0/1)`

The `show_console` built-in system function controls the visibility of a place's console window at runtime. Evaluated as a standard function call rather than a dedicated language keyword, calling `show_console(0)` hides the window; calling `show_console(1)` makes it visible again. Output produced while the window is hidden is still logged and will be replayed when the window reappears.

This is particularly useful for worker places that perform background computations and do not need a visible console:

```
place Worker {
    show_console(0);    // hide console for this place
    num result = 2 ** 10;
    send result to Main as "result";
    show_console(1);    // show console again before exiting
}

place Main {
    num r = receive num from Worker named "result";
    println("Result: %d" % r);
}
```

6.3 Sending Data: `send`

The `send` statement transmits a variable from one place to another through the shared packet network. All four QLang variable types can be sent. The general syntax is:

```
send <variable> to <place> as "<name>";
```

Both the `to` and `as` clauses are optional:

Form	Description
`send var to Bob as "msg";`	Sends the `var` to `Bob` with the packet name `"msg"`. Only `Bob` can receive it, and the receiver can filter by the name.
`send var to Bob;`	Sends the `var` to `Bob` without a packet name. Only `Bob` can receive it.
`send var as "msg";`	Sends the `var` without a specific target. Any place can receive it, and the receiver can filter by the name `"msg"`.
`send var;`	Sends the `var` without a target or name. Any place can pick it up.

The target place can be specified as a bare identifier, a string literal, or a `text` variable containing the place name. However, the `as` packet name must be a string literal and cannot be a variable. The following three statements are equivalent:

```
send x to Bob as "data";
send x to "Bob" as "data";

text target = "Bob";
send x to target as "data";
```

The following example shows the different send forms:

```
place Alice {
    num x = 10;
    text msg = "hello";

    // Full form: target and name
    send x to Bob as "number";

    // Target only, no name
    send msg to Bob;

    // Name only, no target, any place can receive
    send x as "data";
}
```

6.4 Receiving Data: `receive`

The `receive` construct declares a new variable and blocks execution until a matching packet arrives from the network. The general syntax is:

```
<type> <name>[size] = receive [<type>] [from <place>] [named "<name>"];
```

The `receive` keyword accepts zero or more optional filters:

Filter	Description
`type`	An optional type decoration. Note that **this filter is syntactically ignored by the interpreter at runtime**. Packets are instead filtered based strictly on the declared type of the target variable on the left-hand side of the declaration (which determines the quantum vs. classical packet flag).
`from place`	Accept only packets sent from the specified place. The place can be given as a bare identifier or a string literal.
`named "name"`	Accept only packets that were sent with the matching `as` name.

Filters can be combined in any order. If no filters are specified, the receive accepts the first packet whose type and size match the declared variable:

```
place Alice {
    num x = 99;
    send x to Bob;
}

place Bob {
    num x = receive;
    println("Got: %d" % x);
}
```

A more specific receive using all filters:

```
place Sensor {
    num reading = 37;
    send reading to Hub as "temperature";
}

place Hub {
    // Filter by type, source, and name
    num temp = receive num from Sensor named "temperature";
    println("Temperature: %d" % temp);
}
```

6.4.1 Blocking Behaviour

The `receive` call is blocking: the place suspends execution until a matching packet is available. If no matching packet ever arrives, the place will wait indefinitely. This is the intended behavior for synchronization between places.

6.4.2 Array Receives

When receiving an array, the size must be declared with the variable. For example, `num data[3] = receive num from Alice named "data";` expects a packet containing exactly three numeric values.

6.5 Checking Availability: `available`

The `available` expression checks whether a matching packet exists in the network without consuming it or blocking. It returns a boolean value that can be used in conditions. The syntax is:

```
available [<type>] [from <place>] [named "<name>"]
```

All filter arguments are optional. Possible filters are:

Filter	Description
`type`	Check only for packets of the given type (`state`, `obs`, `num`, or `text`).
`from place`	Check only for packets sent from the specified place.
`named "name"`	Check only for packets with the given name.

Unlike `receive`, `available` is non-blocking: it returns immediately with `T` (true) or `F` (false). The packet remains in the network and still needs to be consumed by a `receive` call. A common pattern is to combine `available` with a polling loop:

```
place Alice {
    for i from 0 to 5 {
        num x = (i * 19) % 13;
        wait(1 second); // simulate heavy computation
        send x to Bob as "number";
    }
    text status = "done";
    send status to Bob as "status";
}

place Bob {
    function receive_numbers() {
        num numbers[5];
        for i from 0 to 5 {
            num x = receive num from Alice;
            numbers[i] = x;
        }
        return numbers;
    }

    obs running = T;

    while(running){
        wait(100 ms); // avoid high CPU load
        if (available text named "status"){
            text status = receive text named "status";
            println(receive_numbers());
            running = F;
        }
    }
}
```

In this example, Bob continuously checks whether a `"status"` packet has arrived. Once it has, Bob knows Alice has finished sending numbers and can safely receive them all.

6.6 Packet Matching Rules

The QLang packet network uses a set of matching rules to decide which receiver gets which packet. Every packet carries the following metadata:

Field	Description
`src_id`	Name of the place that sent the packet. Always set automatically.
`target_id`	Name of the intended recipient place. Set only if the sender used `to`.
`msg_id`	Packet name. Set only if the sender used `as`.
`quantum`	Whether the packet carries quantum data (`state`) or classical data.
`size`	Number of elements in the data (1 for scalar values, array length for arrays).

A packet matches a `receive` request when all of the following conditions are satisfied:

1. **Type and size.** The packet's `quantum` flag must match the expected property of the variable being assigned to. The `size` (number of elements) must exactly match the declared size of the receiving variable, unless the variable was declared as a dynamic array with size `[?]`, in which case any size is accepted.
2. **Target.** If the sender specified a `target_id` (using `to`), then the receiver's place name must equal that `target_id`. If the sender did not specify a target, any place can receive the packet.
3. **Source.** If the receiver specified a `from` filter, then the packet's `src_id` must match. If the receiver did not specify `from`, packets from any source are accepted.
4. **Name.** The `msg_id` matching rule has four cases:
   - Both sender and receiver specified a name: the names must be equal.
   - Neither sender nor receiver specified a name: condition is satisfied.
   - Sender specified a name, receiver did not: condition is satisfied (the receiver accepts any name, including named packets).
   - Sender did not specify a name, receiver did: condition fails (the receiver is looking for a specific name that the packet does not have).

When multiple packets match, the oldest one (the first to arrive) is consumed.

6.7 Imports and File Inclusion

QLang allows modularizing code by importing other `.ql` files. Imports must be declared at the global level and support two distinct styles:

- `<< ""path"" >>` – A lexical splicing directive that literally splices the content of the specified file at the current location before parsing. This means any places, variables, and statements in the imported file will be executed as if they were written in the current file.
- `<< "path" >>` – A runtime function import directive that dynamically loads and registers only the function definitions from the specified file during execution. This is useful for building standard function libraries without accidentally executing unwanted side-effects.

6.7.1 Directory Search Prefixes

Both import styles support an optional directory search prefix separated by a colon, for example `<< "lib:file.ql" >>` or `<< ""module:path/to/file.ql"" >>`. When a prefix is specified, the interpreter will search for the requested file within any directory matching the prefix name (e.g., `lib` or `module`) rather than relying strictly on the relative path.

7 Standard Library

This chapter documents the standard library built-in functions and additional operator overloads. These features provide native support for generating random values, mathematical rounding, and specific string/array operations.

7.1 Random Generation

QLang includes globally available built-in functions for generating random values. These functions do not support keyword arguments.

- `seed(value)` — Initializes the internal pseudorandom number generator with the specified numeric seed. If called with no arguments, it returns the current seed.
- `random()` — Returns a pseudorandom number in the range $[0.0, 1.0)$.

7.2 Mathematical Rounding

The rounding functions take a numeric value and an optional `decimals` argument. If `decimals` is omitted or is less than or equal to 0, the function removes any fractional part (returning a whole number). Otherwise, it returns a number rounded to the specified fractional precision.

- `cut(val, decimals=0)` — Truncates the fractional part of `val` towards zero.
- `round(val, decimals=0)` — Rounds `val` to the nearest number. Halfway values are rounded away from zero.
- `floor(val, decimals=0)` — Returns the largest whole number less than or equal to `val`.
- `ceil(val, decimals=0)` — Returns the smallest whole number greater than or equal to `val`.

```
// Set random seed
seed(12345);

// Get a random float [0, 1)
num r = random();

// Math rounding and truncating functions
num val = 12.3456;

num c = cut(val, 2);      // 12.34
num f = floor(val, 1);    // 12.3
num cl = ceil(val, 0);    // 13
num rnd = round(val, 3);  // 12.346
```

7.3 Advanced Operator Overloading

QLang provides implicit operator overloads for `text` variables and arrays. These operators perform data transformations, such as string manipulation and array concatenation.

Expression	Behavior
`text - text`	Removes all occurrences of the characters present in the right-hand string from the left-hand string.
`text * num`	Repeats the text a specified number of times.
`text / text`	Splits the left-hand string by the right-hand separator string, returning an array of parts.
`array + array`	Concatenates two arrays into a single array.
`array + element`	Appends a single element to the end of the array.

```
text word = "hello world";
text clean = word - "o";       // "hell wrld"

text repeated = "ab" * 3;      // "ababab"

text path = "user/local/bin";
text parts[?] = path / "/";    // ["user", "local", "bin"]

num a[?] = [1, 2];
num b[?] = [3, 4];
num combined[?] = a + b;       // [1, 2, 3, 4]
num appended[?] = a + 5;       // [1, 2, 5]
```

8 Debugging

This chapter describes how to troubleshoot QLang programs and use the provided debugging tools effectively.

8.1 The `debug()` Function and `@` Operator

The `debug()` statement is a built-in tool that dumps the internal representation of QLang objects to the console. Every line output by this command is prefixed with `DEBUG` in red text.

It operates in two distinct modes depending on whether the argument is an evaluated expression or a passed reference using the `@` operator:

8.1.1 Expression Mode: `debug(expr)`

When passed a standard expression, `debug()` evaluates it and prints the result's internal shape and value. For example, `debug(2 + 2)` will print the type (`Num`), shape, and the evaluated value (`4`).

For quantum states (`state` variables), `debug(qubit)` provides a deep inspection, revealing:

- **State UID:** The unique identifier of the state object.
- **Quantum History:** A chronological list of all gates and operations applied to that specific state.
- **Stabilizers:** The current stabilizer representation.
- **Raw State Data:** Internal amplitudes (X, Z, and Phase arrays).

8.1.2 Reference Mode: `debug(@identifier)`

Using the `@` prefix operator passes an entity *by reference* without evaluating it. This allows you to inspect the interpreter's metadata for variables and functions.

- **Variable Reference (`debug(@x)`):** Prints the symbol table metadata, including the variable's `Name`, declared `Type`, `Dimensions`, whether it `Is List`, its memory `Index`, and a recursive dump of its stored `Data`.
- **Function Reference (`debug(@myFunc)`):** Prints the function signature, including `Name`, `Parameter Count`, detailed parameter info (names, types, sizes, and default values), `Body Length` (number of statements), and the parsed `Position` in the source file.

8.2 Example

```
place my_place {
    num my_num = 3;
    num my_arr[?] = [42, 2.31, 32];
    text my_text = "test";
    state my_state;

    debug(@my_num);
    debug(@my_arr);
    debug(@my_text);
```

```
    debug(2+2*2);
    debug(@my_state);
}
```

8.3 Packet Logging

When building multi-place programs, a common debugging challenge is identifying deadlocks where a place is stuck in an infinite `receive` block, waiting for a packet that was never sent (or was sent with the wrong target/name).

The `packet_log` built-in system function enables or disables diagnostic logging for network activity in the current place. It is evaluated as a standard function call at runtime rather than a dedicated keyword. Calling `packet_log(1)` turns logging on; `packet_log(0)` turns it off.

When enabled, every `send` and `receive` operation in that place prints detailed information to the interpreter's terminal, including the packet name, source, target, data size, and sequential packet identifier.

```
DEBUG ===== Variable Reference =====
DEBUG Name:       my_num
DEBUG Type:       Num
DEBUG Dimensions: [0]
DEBUG Is List:    False
DEBUG Index:      None
DEBUG Data:
DEBUG   Type: Num
DEBUG   Value: <int.expression.Expression object at 0x000001BCC611D630> (is_float: False)
DEBUG ===== Variable Reference =====
DEBUG Name:       my_arr
DEBUG Type:       Num
DEBUG Dimensions: [3]
DEBUG Is List:    True
DEBUG Index:      None
DEBUG Data:
DEBUG   Type: list
DEBUG   Length: 3
DEBUG   Value: [
DEBUG           Type: Num
DEBUG           Value: <int.expression.Expression object at 0x000001BCC47EE410> (is_float: False)
DEBUG           Type: Num
DEBUG           Value: <int.expression.Expression object at 0x000001BCC46FDA90> (is_float: True)
DEBUG           Type: Num
DEBUG           Value: <int.expression.Expression object at 0x000001BCC46FDA90> (is_float: False)
DEBUG   ]
DEBUG ===== Variable Reference =====
DEBUG Name:       my_text
DEBUG Type:       text
DEBUG Dimensions: [0]
DEBUG Is List:    False
DEBUG Index:      None
DEBUG Data:
DEBUG   Type: Text
DEBUG   Length: 4
DEBUG   Value: test
DEBUG ===== Expression =====
DEBUG Type:  int
DEBUG Shape: None
DEBUG Value:
DEBUG   Type: int
DEBUG   Value: 6
DEBUG ===== Variable Reference =====
DEBUG Name:       my_state
DEBUG Type:       State
DEBUG Dimensions: [0]
DEBUG Is List:    False
DEBUG Index:      None
DEBUG Data:
DEBUG   Type: State
DEBUG   UID: my_place/1
DEBUG   History:
DEBUG           INIT my_place/1
DEBUG   Stabilizers:
DEBUG           D0: +X
DEBUG           S0: +Z
DEBUG   State:
DEBUG           X: [1, 0]
DEBUG           Z: [0, 1]
DEBUG           Phase: [0, 0]
-----[END]-----
```

Figure 1: Output of the debugging example.

9 Examples of Usage

9.1 Example: Quantum Teleportation

Quantum teleportation transfers the state of a qubit from one place to another using a shared entangled pair and two classical bits of communication. The sender measures their qubits and sends the classical results; the receiver applies correction gates based on those results to recover the original state.

```
place Alice {
    println("Alice is preparing to teleport a qubit to Bob...");
    // Prepera Bell pair
    state q1, q2;
    H q1;
    CNOT q1 -> q2;
    // Send one qubit to Bob
    send q2 to Bob as "qubit";
    println("Bell pair prepared. One qubit sent to Bob.");

    // Get one bit of input from user
    state to_send;
    num flip;
    print("Enter a bit to teleport (0 or 1): ");
    input(flip, 0..1);
    println("Input '%d' received. Preparing qubit to teleport..." % flip);

    // 0 = no change, 1 = apply X gate
    if (flip) X to_send;

    // Prepare qubit to teleport
    CNOT to_send -> q1;
    H to_send;

    // Measure both
    obs measurements[2] = measure [to_send, q1];

    // Send measurement results to Bob
    println("Measurement complete: m0=%d m1=%d" % measurements);
    send measurements to Bob as "measurements";
    println("Measurement results sent to Bob.");
}

place Bob {
    // Receive qubit from Alice (half of Bell pair)
    println("Waiting for qubit from Alice...");
```

```
    state q2 = receive state from Alice named "qubit";
    println("Received qubit from Alice.");

    println("Waiting for measurement results from Alice...");
    obs measurements[2] = receive obs from Alice named "measurements";
    println("Received measurement results from Alice.");

    // Apply corrections based on Alice's measurements
    if(measurements[0]) Z q2;
    if(measurements[1]) X q2;

    // q2 should now be in the same state as Alice's original to_send qubit
    println("Teleportation complete.");
    obs orig_state = measure q2;
    println("Original state (0 or 1): %d" % orig_state);
}
```

9.2 Example: Superdense Coding

Superdense coding allows two classical bits of information to be transmitted using only one qubit, provided the sender and receiver share an entangled pair beforehand. The sender encodes the two bits by applying gates to their half of the pair and sends the qubit; the receiver decodes both bits by measuring the pair jointly.

```
place Alice{
    <<"stdlib:utils.ql">>;
    // Prepare entangled pair and send one qubit to Bob
    state qA, qB;
    H qA;
    CNOT qA -> qB;
    send qB to Bob as "qubitB";

    // Get two bits of input from user
    num number;
    print("Enter number 0-3 (two bits): ");
    input(number, 0..3);
    println("Input '%d' received. Preparing qubit to send..." % number);

    obs bits[2] = number >2> obs;
    println("Bits to send: %s" % binary_to_str(bits));

    // Encode the two bits into the qubit
    if (bits[0]) Z qA; // Flip phase for bit 0
    if (bits[1]) X qA; // Flip bit for bit 1
```

```
    // Send the encoded qubit to Bob
    send qA to Bob as "qubitA";
    println("Encoded qubit sent to Bob.");
}

place Bob {
    <<"stdlib:utils.ql">>;
    println("Bob is waiting for qubits from Alice...");
    state qB = receive state from Alice named "qubitB";
    println("Received first qubit from Alice.");
    state qA = receive state from Alice named "qubitA";
    println("Received second qubit from Alice.");

    // Decode the received qubits
    CNOT qA -> qB;
    H qA;
    obs result[?] = measure [qA, qB];
    println("Measurement complete. Decoded bits: %s" % binary_to_str(result));
    println("Decoded number: %d" % (result >> num));
}
```

9.3 Example: Distributed Prime Checker

The following example demonstrates a manager-worker pattern where a `Manager` place distributes tasks to four `Worker` places and collects results. Each worker checks whether a number is prime and sends back the result. To avoid duplicating code, the actual worker logic is written in a separate file (`worker_code.ql`) and included using the `<< ""..."" >>` operator.

The main program (`worker_prime.ql`):

```
place Manager{
    <<"stdlib:utils.ql">>;
    // Manager: dispatch numbers to workers and collect prime results.
    num max_number = 1000; // change as needed
    num primes[?]; // list for collecting primes
    num current_number = 2; // start checking from 2
    // places that will be run as workers
    text workers[?] = ["Worker1", "Worker2", "Worker3", "Worker4"];

    // initial dispatch: send one task to each worker
    iterate workers as worker{
        send current_number to worker as "task";
        current_number++;
```

```
    }

    num runningTasks = #workers; // number of tasks currently running
    num next_worker = 0; // this ensures tasks are split evenly


    while (runningTasks > 0) {
        // collect the result
        num result[2] = receive num named "result"; // [number, isPrime]
        println("Received: %d -> %d" % result);
        if (result[1]) primes += result[0];

        // start new task
        if (current_number <= max_number) {
            text target = workers[next_worker];
            send current_number to target as "task";
            current_number++;
        } else runningTasks--;

        next_worker = (next_worker + 1) % #workers;
    }

    println("Primes found: " + primes);
    println("Total: %d primes." % #primes);

    // Send shutdown signal to workers
    num shutdown_signal = -1;
    iterate workers as worker{
        send shutdown_signal to worker as "task";
    }

    println("Shutdown signals sent to workers. Manager exiting.");
}

// 4 workers, each gets the same code to run
place Worker1{
    <<""worker_code.ql"">>;
}

place Worker2{
    <<""worker_code.ql"">>;
}
```

```
place Worker3{
    <<""worker_code.ql"">>;
}

place Worker4{
    <<""worker_code.ql"">>;
}
```

And the corresponding `worker_code.ql` file that is included by each worker:

```
<<"stdlib:math.ql">>;

show_console(0); // hide console for workers

num processed;

// worker loop that runs until a shutdown signal is received
while (T) {
        num task = receive num from Manager named "task";
    if(task == -1) break; // stop loop at shutdown signal

    println("Worker received task: %d" % task);
        num result[?] = [task, is_prime(task)];
    println("Worker sending result: %d -> %d" % result);
        send result to Manager as "result";
        processed++;
}
println("Worker processed %d tasks." % processed);
println("Worker received shutdown signal. Exiting.");
show_console(1); // show the console again before exiting
```

The Manager dispatches numbers to workers in round-robin fashion. Each worker runs in a loop, receiving tasks, computing results, and sending them back. When the maximum number is reached, the Manager sends a shutdown signal (`-1`) to each worker. The `send`/`receive` pair with `named "task"` and `named "result"` ensures that task packets and result packets are never confused.

9.4 Example: Quantum Key Distribution (BB84)

The following example demonstrates a simplified version of the BB84 quantum key distribution protocol. Alice encodes bits using either the Z-basis (standard) or X-basis (Hadamard), and sends the qubits to Bob. Bob randomly chooses measurement bases. After the transmission, they compare their chosen bases to sift the key.

```
place Alice{
    <<"stdlib:random.ql">>;
    <<"stdlib:utils.ql">>;
```

```
// QUANTUM TRANSMISSION
const num n_qubits = 64; // Amount of transmitted qubits
obs bits[?] = random_bits(count=n_qubits);
obs bases[?] = random_bits(count=n_qubits); // 0=Z 1=X
state q[n_qubits]; // qubits to send

for i from 0 to n_qubits{
    // Encode bits
    if(bits[i]) X q[i];
    // Encode basis
    if(bases[i]) H q[i];
}

println("Number of qubits: %d" % n_qubits);
println("My bits: %s" % binary_to_str(bits));
println("My bases: %s" % binary_to_str(bases, zero="Z", one="X"));

send q to Eve_spy as "qubits"; // send to spy(simulated interception)
println("Qubits sent. Waiting for bases from Bob...");

// EXCHANGE OF BASES (not secret)
obs bob_bases[?] = receive obs from Bob named "bob_bases";
println("Received bases from Bob: %s" % binary_to_str(bob_bases, zero="Z", one="X"));
send bases to Bob as "alice_bases";
println("Sent my bases to Bob.");

// MAKE A SECRET KEY
obs key[?] = [];

for i from 0 to n_qubits{
    if(bases[i] == bob_bases[i]) key += bits[i];
}

println("My secret key: %s" % binary_to_str(key));

// QUBIT ANTI-TAMPER CHECK
const num sample_size = cut(#key / 2);
println("Sample size: %d" % sample_size);

obs key_sample[?] = key[0..sample_size];
println("My sample bits: %s" % binary_to_str(key_sample));
```

```
    send key_sample to Bob as "key_sample";
    println("Sent key sample to Bob. ");

    println("Waiting for error_rate from Bob...");
    num error_rate = receive num from Bob named "error_rate";

    println("Error rate is: %.4f" % error_rate);
    println(error_rate > 0.11 ? "Channel not secure, qubits altered." : "Channel secure.");
}

place Eve_spy{
    <<"stdlib:random.ql">>;
    <<"stdlib:utils.ql">>;

    // T = Eve steals qubits, F = Eve sends qubits untouched
    const obs steal_qubits = T;

    while(T){
        state q[?] = receive state named "qubits";

        // Spying disabled, send as nothing happend
        if(!steal_qubits){
            println("Sending %d qubits untouched." % #q);
            send q to Bob as "qubits";
        }else{
            println("%d qubits stolen." % #q);
            obs guessed_basis[?] = random_bits(count=#q);
            println("Guessed basis: %s" % binary_to_str(guessed_basis, zero="Z", one="X"));

            // Apply guessed basis to stolen qubits
            for i from 0 to #q{
                if(guessed_basis[i]) H q[i];
            }

            // Measure stolen qubits -> original states collapse
            obs measured[?] = measure q;
            println("Measured bits: %s" % binary_to_str(measured));

            // Create the same amount of qubits to send back
            state q_fake[#q];
```

```
            // Encode the "same" information on fake ones
            for i from 0 to #q{
                if(measured[i]) X q_fake[i]; // Encode bit
                if(guessed_basis[i]) H q_fake[i]; // Encode basis
            }

            // Send packet back on it's way
            send q_fake to Bob as "qubits";
        }
    }
}

place Bob{
    <<"stdlib:random.ql">>;
    <<"stdlib:utils.ql">>;

    const num n_qubits = 64; // Amount of qubits
    obs bases[?] = random_bits(count=n_qubits); // 0=Z 1=X

    println("My bases: %s" % (binary_to_str(bases, zero="Z", one="X")));
    println("Waiting for qubits...");

    // RECEIVED QUBITS MEASUREMENT
    state q[?] = receive state named "qubits";

    // Apply Bob's bases
    for i from 0 to n_qubits{
        if(bases[i]) H q[i];
    }

    // Measure all
    obs bits[?] = measure q;
    println("My bits: %s" % binary_to_str(bits));

    // EXCHANGE OF BASES (not secret)
    send bases to Alice as "bob_bases";
    println("Sent bases to Alice. Waiting for bases from Alice...");
    obs alice_bases[?] = receive obs named "alice_bases";
    println("Received bases from Alice: %s" % binary_to_str(alice_bases, zero="Z", one="X"));

    // MAKE A SECRET KEY
    obs key[?] = [];
```

```
    for i from 0 to n_qubits{
        if(bases[i] == alice_bases[i]) key += bits[i];
    }

    println("My secret key: %s" % binary_to_str(key));

    // WAIT FOR KEY SAMPLE
    println("Waiting for key sample from Alice...");
    obs alice_key_sample[?] = receive obs named "key_sample";
    const num sample_size = cut(#key / 2);

    println("Received key sample: %s" % binary_to_str(alice_key_sample));

    // check for errors
    num errors = 0;
    for i from 0 to sample_size{
        if(key[i] != alice_key_sample[i]) errors++;
    }

    num error_rate = errors / sample_size;

    send error_rate to Alice as "error_rate";
    println("Sent error_rate to Alice.");

    println("Error rate is: %.4f" % error_rate);
    println(error_rate > 0.11 ? "Channel not secure, qubits altered." : "Channel secure.");
}
```

E Error codes

When the interpreter encounters any error, it appears in the interpreter terminal formatted inside a red box. Errors are not displayed inside place consoles. After the error is printed out, execution of places is interrupted and interpreter exits.

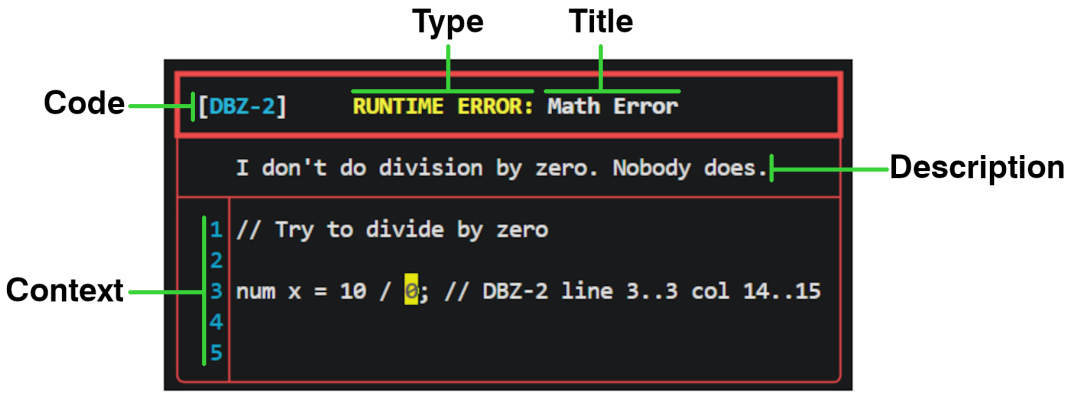

Figure 2: Description of error box elements

Description of each element inside the error box:

- **Type** - When the error happend, can be
  - SYNTAX ERROR - Syntax error was detected in the initial check, code execution did not start.
  - RUNTIME ERROR - Error was discovered during code execution.
- **Title** - Category of the error
- **Code** - Code directly related to specific error. For the full list refer to list below. There are NO codes for syntax errors(code is not displayed).
- **Description** - Short description of what went wrong
- **Context** - Shows few lines above and below. Yellow mark indicates the most likely cause. In rare cases there might be no context to show.

Every error code follows the same format:

`<letters>-<number>`

The letters are compiled of capital letters of the exception that were raised - like a category of what happend. The number narrows it down to the precise case. Below is a table of every set of letter and the section it is described at. Letters are unique to each exception. Some error numbers in the sequence might be missing(eg. 1, 2, 4, 5).

Table 1: List of all error code letters

Letters	Exception	Section
ATE	AssignmentToExpression	??
BEV	BuiltinExpectsValue	??
CFV	CantFindVariable	??
DBZ	DivideByZeroException	??
DQA	DirectQuantumAccess	??
EAV	ExpectedAValue	??
FNF	FileNotFound	??
FR	FunctionRedeclaration	??
I	Internal	??
INI	IndexNotInt	??
IOR	IndexOutOfRange	??
IT	IncompatibleType	??
KANS	KeywordArgumentsNotSupported	??
MA	MissingArg	??
MOZ	ModuloOverZero	??
NFTC	NoFunctionToCall	??
NNV	NetworkNotVariable	??
NPS	NoParentScope	??
ONI	ObsNotIndexed	??
ONS	OperationNotSupported	??
OTM	OperatorTypeMismatch	??
PDNA	PlaceDeclarationNotAllowed	??
PPI	PacketPayloadInvalid	??
PSNI	PacketSizeNotInteger	??
RKA	RepeatedKeywordArg	??
SE	SizeError	??
SM	ShapeMismatch	??
TM	TestMode	??
TMA	TooMuchArguments	??
UA	UnknownArg	??
UCT	UnsupportedConversionType	??
UTU	UnknownTimeUnit	??
VC	VariableCall	??
VR	VariableRedefinition	??
WDNN	WaitDurationNotNumeric	??

E.1 ATE - AssignmentToExpression

See full list: Table **??** –

E.1.1 ATE-1

Left side of AND compound operator is not assignable.

```
1 &= x;
```

E.1.2 ATE-2

Left side of OR compound operator is not assignable.

```
1 |= x;
```

E.1.3 ATE-3

Left side of divide compound operator is not assignable.

```
1 /= x;
```

E.1.4 ATE-4

Left side of substraction compount operator is not assignable.

```
1 -= x;
```

E.1.5 ATE-5

Left side of modulo/formatting compount operator is not assignable.

```
1 %= x;
```

E.1.6 ATE-6

Left side of multiplication compound operator is not assignable.

```
1 *= x;
```

E.1.7 ATE-7

Left side of addition compound operator is not assignable.

```
1 += x;
```

E.1.8 ATE-8

Left side of power compound operator is not assignable.

```
1 **= x;
```

E.1.9 ATE-9

Pre-decrement operator target variable is not assignable.

```
--5;
```

E.1.10 ATE-10

Post-decrement operator target variable is not assignable.

```
5--;
```

E.1.11 ATE-11

Pre-increment operator target variable is not assignable.

```
++5;
```

E.1.12 ATE-12

Pre-increment operator target variable is not assignable.

```
5++;
```

E.1.13 ATE-13

Attempted assign to expression.

```
print(10 = 0);
```

E.2 BEV - BuiltinExpectsValue

See full list: Table **??** –

E.2.1 BEV-1

Built-in function `num()` requires one argument: number or numeric string.

```
num("abc");
```

E.2.2 BEV-2

Built-in function `parent()` requires one argument: variable name or nested parent call.

```
parent(5);
```

E.2.3 BEV-3

Built-in function `seed()` requires one argument: a number.

```
seed("42");
```

E.2.4 BEV-4

Import tags `<<  >>` require a file path as string.

```
<<"path/to/file.ql">>;
<<""path/to/file.txt"">>;
```

E.2.5 BEV-5

Built-in function `packet_log()` requires one argument: number 0 or 1.

```
packet_log(2);
```

E.2.6 BEV-6

Built-in function `show_console()` requires one argument: number 0 or 1.

```
show_console(2);
```

E.2.7 BEV-7

Built-in functions `cut()`, `round()`, `floor()` and `ceil()` requires first argument to be a number.

```
cut("string");
```

E.2.8 BEV-8

Built-in functions `cut()`, `round()`, `floor()` and `ceil()` requires the second argument to be an integer number of decimal places.

```
cut(2.5, "123");
```

E.3 CFV - CantFindVariable

See full list: Table **??** –

E.3.1 CFV-1

Trying to read size of a variable that does not exist.

```
println(#non_existing_var);
```

E.3.2 CFV-2

Trying to read type of a variable that does not exist.

```
println($non_existing_var);
```

E.3.3 CFV-3

Trying to get variable/value that does not exist.

```
println(non_existing_var);
```

E.3.4 CFV-4

Parent scope does not contain the requested variable.

```
num y = 0;
if(T){
    num x = 0;
    println(x);
    println(parent(x)); // <- error here
}
----------------------------------------
num y = 0;
if(T){
    num x = 0;
    println(x);
    println(^x); // <- error here
}
```

E.3.5 CFV-5

Built-in function `input()` called with a variable that does not exist.

```
input(non_existing_var);
```

E.3.6 CFV-7

Tried to get reference of a variable that does not exist.

```
debug(@non_existing_var);
```

E.3.7 CFV-8

Operator `reset` used on variable that does not exist.

```
reset non_existing_var;
```

E.3.8 CFV-10

Operator `const` used to make a non existing variable const.

```
const non_existing_var;
```

E.4 DBZ - DivideByZeroException

See full list: Table **??** –

E.4.1 DBZ-1

Right side of division compound operator is 0 (division by 0).

```
num x = 10;
x /= 0;
```

E.4.2 DBZ-2

Right side of division operator is 0 (division by 0).

```
num x = 10 / 0;
```

E.5 DQA - DirectQuantumAccess

See full list: Table **??** –

E.5.1 DQA-1

Attempted direct read of quantum state in parenthness.

E.5.2 DQA-2

Tried getting the value of variable of type `state`.

```
state x;
print(x);
```

E.5.3 DQA-3

Variable of type `state` used as argument for built-in function `input()`.

```
state x;
input(x);
```

E.5.4 DQA-4

Operator `reset` applied to a variable of type `state`

```
state s;
reset s;
```

E.5.5 DQA-5

Tried to assign something to a variable of type `state`.

```
state x;
x = 10;
```

E.5.6 DQA-6

Tried to assign-expression something to a variable of type `state`

```
state x;
print(x = 10);
```

E.5.7 DQA-7

Tried to assign initial value to a variable of type `state`.

```
state x = 10;
```

E.6 EAV - ExpectedAValue

See full list: Table **??** –

E.6.1 EAV-1

Expression must be a string variable or text.

E.6.2 EAV-2

Expected a string or text expression during string/text concatenation or formatting.

E.6.3 EAV-3

Expected a list expression during list operations or binary conversion

E.7 FNF - FileNotFound

See full list: Table **??** –

E.7.1 FNF-1

Imported file path does not exist.

```
<< "does/not/exist.ql" >>
```

E.8 FR - FunctionRedeclaration

See full list: Table **??** –

E.8.1 FR-1

Trying to declare a function that already exists.

```
function foo(){

}

function foo(){ // <- error here

}
```

E.9 I - Internal

See full list: Table **??** –

Errors listed below are related to internal interpreter faults, not to executed code!

E.9.1 I-1

Quantum network used before initialized.

E.9.2 I-2

Receive attempted before quantum network was initialized.

E.9.3 I-3

Available attempted before quantum network was initialized.

E.9.4 I-4

Quantum operation requires a state variable.

E.9.5 I-5

Select exactly one state element.

E.9.6 I-6

Quantum operation requireds a state or list of states.

E.10 INI - IndexNotInt

See full list: Table **??** –

E.10.1 INI-2

Starting index in range index is not an integer.

```
num numbers[?] = [1, 2, 3, 4, 5, 6];
num nope[?] = numbers[1.5..0]; // <- error here
```

E.10.2 INI-3

Ending index in range index is not an integer.

```
num numbers[?] = [1, 2, 3, 4, 5, 6];
num nope[?] = numbers[1..2.5]; // <- error here
```

E.10.3 INI-4

One of the elements on list index is not an integer.

```
num numbers[?] = [1, 2, 3, 4, 5, 6];
num nope[?] = numbers[[0, 2.5, 4]]; // <- error here
```

E.10.4 INI-5

Index is not an integer.

```
num numbers[?] = [1, 2, 3, 4, 5, 6];
num nope[?] = numbers[2.5]; // <- error here
```

E.11 IOR - IndexOutOfRange

See full list: Table **??** –

E.11.1 IOR-1

Index out of range during array or text access.

```
num numbers[?] = [1, 2, 3, 4, 5, 6];
num x = numbers[10]; // <- error here
```

E.12 IT - IncompatibleType

See full list: Table **??** –

E.12.1 IT-1

Function was given a wrong type of argument.

```
function foo(num arg1, num arg2){
    println(arg1);
    println(arg2);
}

foo(arg1=5, arg2="10"); // <- error here
```

E.13 KANS - KeywordArgumentsNotSupported

See full list: Table **??** –

E.13.1 KANS-1

Built-in function `num()` called with named arguments.

```
num(val=10);
```

E.13.2 KANS-2

Built-in function parent() called with named arguments.

```
parent(val=10);
```

E.13.3 KANS-3

Built-in function parent() called with named arguments.

```
parent(val=10);
```

E.13.4 KANS-4

Built-in function seed() called with named arguments.

```
seed(val=10);
```

E.13.5 KANS-5

Built-in function random() called with named arguments.

```
random(val=10);
```

E.13.6 KANS-6

Internal function `__ql_import_source__` called with named args.

```
random(val=10);
```

E.13.7 KANS-7

Built-in function `packet_log()` called with named arguments.

```
packet_log(val=10);
```

E.13.8 KANS-8

Built-in function `show_console()` called with named arguments.

```
show_console(val=10);
```

E.13.9 KANS-9

Built-in function `cut()`, `round()`, `floor()` or `ceil()` called with named arguments.

```
cut(val=10);
```

E.14 MA - MissingArg

See full list: Table **??** –

E.14.1 MA-1

Function called without a required argument.

```
function foo(num arg1, num arg2){
    println(arg1);
    println(arg2);
}

foo(5); // <- error here
```

E.15 MOZ - ModuloOverZero

See full list: Table **??** –

E.15.1 MOZ-1

Left side of modulo operator is not string/text and right side is 0.

```
10 % 0;
```

E.15.2 MOZ-2

Left side of modulo compound operator is not a string/text and right side is 0.

```
num n = 5;
n %= 0;
```

E.16 NFTC - NoFunctionToCall

See full list: Table **??** –

E.16.1 NFTC-1

Function that does not exist got called.

```
foo(5);
```

E.17 NNV - NetworkNotVariable

See full list: Table **??** –

E.17.1 NNV-1

Trying to send something that is not a variable.

```
send 5;
```

E.18 NPS - NoParentScope

See full list: Table **??** –

E.18.1 NPS-1

Built-in function `parent()` ran out of scopes to go up into.

```
num x = 0;
if(T){
    num x = 1;
    parent(parent(x)); // <- error here
}
```

E.18.2 NPS-2

Parent operator ran out of scopes to go up into.

```
num x = 0;
if(T){
    num x = 1;
    ^^x = 10; // <- error here
}
```

E.19 ONI - ObsNotIndexed

See full list: Table **??** –

E.19.1 ONI-1

Variable of type obs used with an index inside input()

```
obs x;
input(x[3]); // <- error here
```

E.20 ONS - OperationNotSupported

See full list: Table **??** –

E.20.1 ONS-1

Variable of type `state` cannot be made const.

```
const state x; // <- this is not supported
state y;
const y; // <- this is not too
```

E.20.2 ONS-2

Variable type not supported.

```
unknown_type x = 10;
```

E.20.3 ONS-3

`const` can not be used as variable type.

```
const x = 15;
```

E.20.4 ONS-4

Binary conversion to number requires a list of bits at input.

```
num x = "123" >> num;
```

E.20.5 ONS-5

Binary conversion input needs to be a list of integers.

```
num x = [1.5, 0, 0] >> num;
```

E.20.6 ONS-7

Binary conversion input list needs to be a list of integers 0 or 1.

```
num x = [2, 2, 2, 0] >> num;
```

E.20.7 ONS-8

Binary conversion from a number requires a non-negative integer as the input value.

```
obs x[?] = "123" >10> obs;
```

E.20.8 ONS-11

Float cast requires a number.

```
text s = "cos";
num a = s.0;
```

E.21 OTM - OperatorTypeMismatch

See full list: Table **??** –

E.21.1 OTM-1

Unsupported types for comparision with greater or equal operator **>=**

```
2.5 >= "string";
```

E.21.2 OTM-2

Unsupported types for comparision with greater operator **>**

```
2.5 > "string";
```

E.21.3 OTM-3

Unsupported types for comparision with less or equal operator **<=**

```
"string" <= 2.5;
```

E.21.4 OTM-4

Unsupported types for comparision with less operator **<**

```
"string" < 2.5;
```

E.21.5 OTM-5

Unsupported types for comparision with equal operator **==**

E.21.6 OTM-6

Unsupported types for comparision with not equal operator **!=**

E.22 PDNA - PlaceDeclarationNotAllowed

See full list: Table **??** –

E.22.1 PDNA-1

Imported file contains place declarations.

```
<<"to_import.ql">>;
```

```
// File to_import.ql
place ShouldNotBeHere{


}
```

E.23 PPI - PacketPayloadInvalid

See full list: Table **??** –

E.23.1 PPI-1

Quantum state cannot be deserialized (invalid/corrupted packet). Most likely internal fault.

E.23.2 PPI-2

Received packet does not match type state. Most likely internal fault.

E.24 PSNI - PacketSizeNotInteger

See full list: Table **??** –

E.24.1 PSNI-1

Recevie packet size must be an integer.

```
num x[2.5] = receive num;
```

E.24.2 PSNI-2

Receive packet size must be positive.

```
num x[-2] = receive num;
```

E.24.3 PSNI-3

Receive matched a packet with negative or not integer size. Most likely internal fault.

E.25 RKA - RepeatedKeywordArg

See full list: Table **??** –

E.25.1 RKA-1

Function called with one or more named arguments repeated.

```
function foo(num arg1, num arg2){
    println(arg1);
    println(arg2);
}

foo(arg1=5, arg2=10, arg1=15);
```

E.26 SE - SizeError

See full list: Table **??** –

E.26.1 SE-1

Array declaration size is not an integer.

```
num numbers[2.5];
```

E.26.2 SE-2

Array declaration size is 0 or negative.

```
num numbers[-5];
```

E.26.3 SE-3

Binary conversion bit width is 0 or negative.

```
obs bits[?] = 42 >-10> obs;
```

E.27 SM - ShapeMismatch

See full list: Table **??** –

E.27.1 SM-1

Array assigned/modified with a value of different shape.

```
num a[5];
num b[3] = a; // <- error here
```

E.28 TM - TestMode

See full list: Table **??** –

E.28.1 TM-1

`input()` called while interpreter was running in TEST_MODE. Interpreter enters test mode when int.run python module is executed with additional TEST_MODE argument after the .ql file path. Test mode should only be used for internal interpreter testing.

E.29 TMA - TooMuchArguments

See full list: Table **??** –

E.29.1 TMA-1

Function call was given more args than it can take.

```
function foo(num arg1, num arg2){
    println(arg1);
    println(arg2);
}

foo(5, 10, 15); // <- error here
```

E.29.2 TMA-2

Built-in function `num()` called with more than 1 argument.

```
num x = num(2.5, 2.5);
```

E.29.3 TMA-3

Built-in function `parent()` called with more than 1 argument.

```
parent(1, 2, 3);
```

E.29.4 TMA-4

Built-in function `seed()` called with more than 1 argument.

```
seed(1, 2, 3);
```

E.29.5 TMA-5

Built-in function `random()` called with more than 0 arguments.

```
random(1, 2, 3);
```

E.29.6 TMA-6

Internal function `__ql_import_source__` called with more than 1 argument. This function should not be called manually.

E.29.7 TMA-7

Built-in function `packet_log()` called with more than 1 argument.

```
packet_log(1, 2, 3);
```

E.29.8 TMA-8

Built-in function `show_console()` called with more than 1 argument.

```
show_console(1, 2, 3);
```

E.29.9 TMA-9

Built-in function `cut()`, `round()`, `floor()` or `ceil()` called with 0 or more than 2 arguments.

```
cut(1, 2, 3);
```

E.30 UA - UnknownArg

See full list: Table **??** –

E.30.1 UA-1

Function called with unknown named argument.

```
function foo(num arg1, num arg2){
    println(arg1);
    println(arg2);
}

foo(arg1=5, arg2=10, arg3=15);
```

E.31 UCT - UnsupportedConversionType

See full list: Table **??** –

E.31.1 UCT-1

Binary conversion output type is not supported.

```
text s = 42 >10> text;
```

E.32 UTU - UnknownTimeUnit

See full list: Table **??** –

E.32.1 UTU-1

Unknown time unit used in `wait()`

```
wait(12 inches);
```

E.33 VC - VariableCall

See full list: Table **??** –

E.33.1 VC-1

Attempted variable call like it is a function.

```
num foo = 0;
foo(5); // <- error here
```

E.34 VR - VariableRedefinition

See full list: Table **??** –

E.34.1 VR-1

Variable with that name already exists.

```
num x = 5;
num x = 10; // <- error here
```

E.35 WDNN - WaitDurationNotNumeric

See full list: Table **??** –

E.35.1 WDNN-1

`wait()` duration is not numeric.

```
wait("two" seconds);
```

R Implementation Report

This chapter provides a technical description of the interpreter's internal architecture, the use of ANTLR 4 for parsing, the stabiliser-based quantum simulation engine, the multiprocessing model, and the testing framework.

R.1 Project Structure

The repository is organised as follows:

```
QLang/
  antlr/
    QLang.g4              ANTLR4 combined grammar
    antlr-4.13.2-complete.jar
  int/                    Interpreter source (Python)
    QLang/                Generated lexer/parser (output of ANTLR)
    context/              Handler modules for AST node types
      control/            for, while, iterate
      expression/         variables, numbers, lists, booleans
      function/           declaration and invocation
      io/                 print, println, debug, input
      math/               arithmetic and logical operators
      operator/           comparisons, increment, cast, reset
      statement/          statement dispatch, break/continue
      time/               wait
      variable/           declaration, assignment
      network.py          send / receive / available
      quantum.py          gate application and measurement
    exception/            39 runtime exception classes
    operations/           arithmetic helpers
    sim/                  Quantum simulator subsystem
      data/               Data structures (gates, graph, request, ...)
      exceptions/         Simulator-specific exceptions
      simulator.py        Stabiliser tableau core
      server.py           QuantumServer (separate process)
      client.py           QuantumClient (per-place proxy)
    run.py                Entry point
    place.py              Place class (execution unit)
    places.py             Place partitioning logic
    network.py            QuantumNetwork (packet pool)
    scope.py              Scope manager (variable scoping)
    variable.py           Variable class
    expression.py         Expression and ValueResolver
    function.py           Function / FunctionParam
    state.py              State (qubit handle)
    obs.py                Obs / ObsRegister
    num.py                Num
    text.py               Text
    imports.py            Import preprocessor
    consts.py             ANTLR context type aliases
    console.py            Per-place console window
    script_errors.py      Error display (coloured box)
    logger.py             Logging subsystem
  stdlib/                 Standard library (.ql files)
  tests/                  Test framework and test suites
  highlighter/            VS Code syntax highlighting extension
  pyproject.toml          Project metadata and dependencies
  run.bat                 Build-and-run script
```

R.1.1 Python Dependencies

The project is managed by the `uv` tool. Dependencies declared in `pyproject.toml`:

Package	Version	Role
`antlr4-python3-runtime`	$\geq 4.13.2$	Runtime for the generated parser
`colorama`	$\geq 0.4.6$	Coloured terminal output
`pygls`	$\geq 1.3.0$	Language Server Protocol support

The required Python version is $\geq$ **3.14**.

R.2 ANTLR 4 Grammar and Parser Generation

R.2.1 Grammar File

The entire language is defined in a single combined grammar file `antlr/QLang.g4`. A combined grammar means the lexer and parser rules coexist in one file.

Parser generation is executed automatically on every interpreter run by the `run.bat` script:

```
java -jar antlr-4.13.2-complete.jar \
     -Dlanguage=Python3 QLang.g4 \
     -o ..\int\QLang\ -visitor
```

The generated files (`QLangLexer.py`, `QLangParser.py`, `QLangVisitor.py`, and token files) are placed in `int/QLang/`.

R.2.2 Top-Level Grammar Rules

```
program : topLevelItem* EOF ;

topLevelItem
    : placeDecl       // place declaration
    | functionDecl    // function (goes to global place)
    | statement       // statement (goes to global place)
    ;
```

A program is a sequence of top-level items: place declarations, function declarations, or statements. Items that are not wrapped in a `place` block belong to the implicit `global` place.

R.2.3 Operators

ANTLR 4 resolves the ambiguity of left-recursive expressions by assigning higher precedence to alternatives listed earlier in the `expr` rule. The effective precedence table (highest first):

#	Operators	Label
1	`!x`, `-x`, `+x`	Unary
2	`++x`, `--x`	Pre-increment / decrement
3	`x++`, `x--`	Post-increment / decrement
4	`x.0` (float cast)	FloatCastExpr
5	`= += -= *= /= %= **= &= |=`	Assignment / compound
6	`**`	Exponentiation
7	`* / %`	Multiplicative
8	`+ -`	Additive
9	`>>`	Binary-from-list
10	`< > <= >=`	Relational
11	`== !=`	Equality
12	`&&`	Logical AND
13	`||`	Logical OR
14	`$x`	Type query
15	`c ? a : b`	Ternary

R.2.4 Lexer Highlights

- **Identifiers** use the Unicode pattern `[\p{L}_][\p{L}\p{N}_]*`, supporting non-ASCII characters (including Polish letters).
- **Numbers**: decimal, hexadecimal (`0x…`), binary (`0b…`), and scientific notation (`1.5e10`).
- **Strings**: three quoting styles — double quotes, single quotes, and backticks.
- **Comments**: line (`//`) and block (`/* …*/`).

R.2.5 Custom Error Listener

The default ANTLR 4 error listener is replaced by a custom class, `QLangErrorListener` (in `int/run.py`). It analyses the parser's context stack (`_ctx`) to produce user-friendly error messages instead of raw ANTLR output. Over 50 context-specific error patterns are defined, each mapping a combination of parser context and offending token to a human-readable title and explanation (e.g. `LeftOpen`, `BrokenIf`, `NoVariable`, `BadGate`).

R.3 Interpreter Pipeline

The full lifecycle of a QLang program is:

1. **Read** the `.ql` file (UTF-8).
2. **Preprocess imports**: the preprocessor in `imports.py` resolves the `<< "path" >>` and `<< ""path"" >>` directives before lexing.
3. **Lex and parse** with the ANTLR 4-generated lexer and parser.
4. **Partition into places**: `divideIntoPlaces()` in `places.py` walks the top-level items and groups them by place name. For each place, the source text of every member is stored together with its original line number.
5. **Start the quantum server**: a dedicated `multiprocessing.Process` running `QuantumServer.run()`.
6. **Create the packet network**: a shared `QuantumNetwork` backed by a `multiprocessing` manager object.
7. **Launch each place** as a separate OS process.
8. **Wait** for all place processes to terminate.
9. **Shut down** the quantum server.

R.3.1 Re-Parsing Inside Place Processes

Because ANTLR 4 parse trees are not serialisable with Python's `pickle`, they cannot be shared across `multiprocessing` boundaries. The interpreter works around this by storing each place's code as plain text and **re-parsing** it inside the child process:

```
# inside Place.process_target()
place_text = "place " + self.name + "{\n"
for block in self.blocks:
    place_text += block.text + "\n"
place_text += "};"

input_stream = InputStream(place_text)
lexer  = QLangLexer(input_stream)
```

```
parser = QLangParser(CommonTokenStream(lexer))
tree   = parser.placeDecl()

for member in tree.placeMember():
    self.handle_block(member.getChild(0), None)
```

R.4 Handler Dispatch Mechanism

Although ANTLR 4 generates a Visitor base class, the interpreter does not use it. Instead, a type-to-handler dictionary is defined in `Place.handle_block()`:

```
HANDLERS = {
    TerminalCtx:          handle_terminal,
    StatementCtx:         handle_statement,
    VarDeclCtx:           handle_variable_declaration,
    GateStmtCtx:          handle_gate_statement,
    MeasureExprCtx:       handle_measure_expr,
    IfStmtCtx:            handle_if,
    ForStmtCtx:           handle_for,
    WhileStmtCtx:         handle_while,
    ...                   # approximately 70 mappings total
}
```

For every AST node, the method iterates over the keys of `HANDLERS` and checks `isinstance(block, type)`. When a match is found, the corresponding handler function is called with the signature `handler(self, block, parent, pos)`.

Each handler is imported from a dedicated module inside the `int/context/` package. Type aliases for ANTLR context classes (e.g. `VarDeclCtx = QLangParser.VarDeclContext`) are centralised in `int/consts.py`.

R.5 Internal Type System

The interpreter distinguishes two layers of types:

Layer	Types	Purpose
Primitive	`int`, `float`, `bool`, `string`, `list`	Internal evaluation types carried by `Expression`
Variable wrapper	`Obs`, `Num`, `State`, `Text`, `any`	Language-level types, each backed by a dedicated Python class

R.5.1 Expression

`Expression` (`int/expression.py`) is the main value carrier returned by handlers. It stores a `type` string, a `value`, and an optional `shape`. The class overloads all Python arithmetic and comparison dunder methods with automatic type promotion following the chain: `bool` → `int` → `float` → `string`.

R.5.2 ValueResolver

`ValueResolver` provides static helper methods:

- `extract_raw_value(v)` — iteratively unwraps nested `Expression`/`Variable`/wrapper objects down to a Python primitive.
- `resolve_for_operation(v)` — unwraps to the level needed for arithmetic, preserving `Obs` / `Num` / `Text` wrappers that define their own operators.

- `wrap_value(type, v)` / `wrap_list(…)` — wraps raw values back into the appropriate class.

R.5.3 Variable

`Variable` (`int/variable.py`) supports: multi-dimensional arrays, dynamic arrays, constants, three indexing modes (`SimpleIndex`, `RangeIndex`, `ListIndex`), character-level indexing into `Text`, and reset-to-initial-value.

R.6 Scope System

The scope system manages variable visibility and lifetime across all control-flow constructs. It is implemented by two classes in `int/scope.py`: `Scope` and `ScopeManager`.

R.6.1 Scope Data Structure

Each `Scope` instance holds:

Field	Description
`vars`	Dictionary mapping variable names to their value objects (`Variable`, `Num`, `Function`, …).
`pos`	Source-code position (`ScriptErrors.Position`) where this scope was created.
`type`	String label identifying what construct created the scope: `"block"`, `"for"`, `"for_iteration"`, `"while"`, `"if"`, `"iterate"`, `"iterate_iteration"`, `"function"`.
`parent`	Reference to the enclosing `Scope`, or `None` for the global scope.

Scopes form a singly-linked list (child → parent). The global scope is always at the root and has `parent = None`.

R.6.2 ScopeManager

`ScopeManager` maintains a single pointer `current` to the innermost active scope. It exposes the following operations:

Method	Behaviour
`push(pos, type)`	Creates a new `Scope` whose parent is the current scope, and makes it current.
`pop()`	Restores `current` to the parent scope, effectively destroying the inner scope and all of its variables. Raises an exception when attempting to pop the global scope.
`get(name)`	Walks the scope chain from `current` upward and returns the first matching variable.
`exists(name)`	Same walk, returns a boolean.
`create(name, value)`	Inserts a variable into the *current* scope only (may shadow an outer variable of the same name).
`set(name, value)`	Walks upward to find an existing variable and overwrites it; if not found, creates it in the current scope.
`assign(name, value)`	Walks upward and overwrites only if found (never creates).
`modify(name, func)`	Walks upward, calls `func(old_value)` and stores the result back in the same scope.
`delete(name)`	Walks upward and removes the first matching variable.

The distinction between `create` and `set`/`assign` is critical: variable declarations use `create`, which always inserts into the current scope and enables shadowing, while assignments use `set` or `assign`, which walk upward to find the existing binding.

R.6.3 Scope Lifecycle in Control Structures

Every control-flow construct that introduces a new lexical context pushes a scope on entry and pops it on exit. The scope type label differs per construct:

- **For loop** (`handle_for_loop`): pushes a `"for"` scope containing the counter variable, then pushes an additional `"for_iteration"` scope for each iteration (cleared after every iteration to discard loop-body variables). Both scopes are popped on loop exit.
- **While loop** (`handle_while`): pushes a `"while"` scope for each iteration and pops it at the end of the iteration.
- **Iterate loop** (`handle_iterate`): pushes an `"iterate"` scope for the element and optional index variable, then pushes an `"iterate_iteration"` scope per iteration, identical to the for loop pattern.
- **If statement** (`handle_if`): pushes an `"if"` scope before executing the chosen branch and pops it afterward.

All handlers use `try/finally` to guarantee that scopes are popped even when `break`, `continue`, or exceptions interrupt normal flow.

R.6.4 Functions and Closures

Functions interact with scopes in two phases:

1. **Declaration** (`handle_function_declaration`): the current scope at the point of the `function` keyword is captured and stored in the `Function` object as `closure_scope`. The function is then inserted into the current scope via `scopes.create()`.

2. **Invocation** (`handle_function_call`): a new `Scope` is created with `parent = closure_scope` (not the caller's scope). Function parameters are added as variables in this new scope. The `ScopeManager.current` pointer is temporarily redirected to the new scope; after execution (whether normal or via `FunctionReturn`), the original scope is restored.

This design means functions act as **closures**: they see the variables from the scope in which they were *defined*, not from the scope in which they are *called*.

R.6.5 Parent Operator

The parent operator `^` (`handle_operator_parent`) allows explicit scope navigation. The handler counts the number of leading `^` characters, walks that many links up the parent chain, and looks up the variable *only in that target scope* (without the automatic upward walk). This is useful for reaching a shadowed outer variable. If the requested depth exceeds the available scope chain, a `NoParentScopeException` is raised.

R.7 Place System

The place system implements QLang's concurrency model, where each `place` declaration becomes an independent OS process with its own scope tree, console window, and quantum client.

R.7.1 Place Partitioning (`divideIntoPlaces`)

The function `divideIntoPlaces()` in `int/places.py` walks the parsed AST's top-level items and partitions them into named groups:

1. A `global` place is created unconditionally at position $(0, 0)$.
2. For each `PlaceDeclContext` node, the place name is validated (Unicode letters, digits, underscores; not `"global"`) and checked for uniqueness. A new `Place` is created and each `placeMember` child is stored as source text together with its original line number.
3. Top-level `FunctionDecl` and `Statement` nodes that are not inside any `place` block are appended to the `global` place.

The source text of each member is extracted via the ANTLR token stream (`stream.getText(start_index, stop_index)`) to preserve the original formatting. The original line number is stored alongside the text so that error messages in child processes can point back to the correct location in the source file.

R.7.2 Place Class

Each `Place` instance (`int/place.py`) encapsulates:

Field	Description
`name`	Place name (e.g. `"Alice"`, `"global"`).
`block`	List of source-text fragments (with line numbers) that constitute this place's code.
`scopes`	A `ScopeManager` instance — each place has its own, independent scope tree.
`console`	A `Console` instance for I/O, opened as a separate CMD window.
`quantum_client`	A `QuantumClient` proxy to the shared quantum server.
`quantum_network`	Reference to the shared `QuantumNetwork` for `send`/`receive`.
`script_errors`	`ScriptErrors` instance for error display.

R.7.3 Per-Place Re-Parsing and Execution

Because ANTLR 4 parse trees are not serialisable with Python's `pickle`, they cannot be transferred across `multiprocessing` boundaries. Each place therefore stores its code as plain text and re-parses it inside the child process in `Place.process_target()`:

1. The stored text fragments are concatenated into a synthetic `place Name { …};` string, with blank lines inserted to preserve the original line numbering.
2. A fresh ANTLR lexer and parser produce a new parse tree.
3. Each `placeMember` child is dispatched to `handle_block()`, the central handler-dispatch method.

The handler dispatch uses a dictionary of $\sim$70 ANTLR context types mapped to handler functions (described in Section **??**). Each handler receives the place instance (`self`), the AST node, the parent node, and the source position.

R.7.4 Multiprocessing Execution

The `Places.run()` method iterates over all registered places and calls `Place.run()` on each one. `Place.run()` creates a `multiprocessing.Process` with `process_target` as the entry point. All places start concurrently and run in true parallel (bypassing Python's GIL).

The main process then calls `Places.wait_for_end()`, which joins every place process. The `global` place is skipped if it contains no code.

R.7.5 Inter-Place Isolation

Each place is fully isolated:

- **Scope isolation**: every place creates its own `ScopeManager` with its own global scope. Variables and functions declared in one place are invisible to other places.
- **Console isolation**: every place opens a separate CMD window via TCP socket. I/O calls (`print`, `println`, `input`) are routed to the place's own console.
- **Communication**: the only way to exchange data between places is through the shared `QuantumNetwork` packet pool, using the `send`/`receive`/`available` constructs.

R.8 Quantum Simulator

R.8.1 Client–Server Architecture

The simulator runs in a dedicated OS process. Communication between places and the simulator uses `multiprocessing.Queue`:

- One shared command queue — all places enqueue `QuantumRequest` objects.
- One response queue per place — the server returns `QuantumResponse` to the correct caller.

The server runs an infinite loop that dequeues requests, dispatches them to the `QuantumSimulator` instance, and enqueues the response. Ten command types are supported: `CREATE`, `LIST`, `STAB`, `STATES`, `HISTORY`, `GATE`, `MEASURE`, `REMOVE`, `ALIAS`, `SEED`.

R.8.2 Stabiliser Tableau

The core simulation engine (`int/sim/simulator.py`) implements the stabiliser tableau formalism. For n qubits, the data structure consists of:

- **X**: a $2n \times n$ bit matrix (X components of Pauli operators).
- **Z**: a $2n \times n$ bit matrix (Z components).
- **phase**: a $2n$-element vector (values 0 or 2, representing signs $+1$ and -1).

Rows $0 \ldots n{-}1$ are destabilisers, rows $n \ldots 2n{-}1$ are *stabilisers*. Each Pauli operator is encoded as a pair of bits (x, z):

Pauli	x	z
I	0	0
X	1	0
Y	1	1
Z	0	1

A precomputed Pauli multiplication table stores the product and phase offset for every pair of Pauli operators, enabling $O(1)$ row-multiplication steps.

R.8.3 Gate Implementation

All supported gates belong to the Clifford group and are implemented as direct bit manipulations on the tableau:

Gate	Tableau transformation
H	Swap columns $X \leftrightarrow Z$; adjust phase if both bits are 1.
S	$Z_{r,c} \oplus= X_{r,c}$; adjust phase.
X	Flip phase if $Z_{r,c} = 1$.
Y	Flip phase if $X_{r,c} \oplus Z_{r,c} = 1$.
Z	Flip phase if $X_{r,c} = 1$.
CNOT(c,t)	$X_{r,t} \oplus= X_{r,c}$; $Z_{r,c} \oplus= Z_{r,t}$; phase correction.
CZ	Decomposed as $H(t) \to$ *CNOT*$(c,t) \to H(t)$.

The `swap` gate is decomposed at interpreter level as three consecutive CNOT gates.

Because the stabiliser formalism only supports Clifford gates, non-Clifford gates (e.g. T, Toffoli, arbitrary rotations) cannot be simulated by this engine. This is a deliberate trade-off: Clifford circuits can be simulated in $O(n^2)$ time per gate, whereas a general state-vector simulation would require $O(2^n)$ memory.

R.8.4 Measurement

Measurement of qubit q proceeds as follows:

1. Search stabiliser rows ($n \dots 2n-1$) for a row where $X_{r,q} = 1$.
2. If found:
   - The outcome is sampled uniformly (0 or 1). If a per-prefix seed was set via `seed()`, a deterministic `random.Random(seed)` is used.
   - The tableau is updated to reflect the post-measurement projective collapse.
3. If not found:
   - The outcome is computed from the destabiliser rows.
4. The entanglement graph calls `split(q)` to remove q from its entanglement group.

R.8.5 Entanglement Graph

The class `EntaglementGraph` (`int/sim/data/graph.py`) implements a **Union–Find** structure with path compression and union-by-rank. It tracks which qubits are entangled:

- `union(a, b)` — called when a two-qubit gate links a and b.
- `split(x)` — called after measurement, disconnects x from its group.
- `connected(a, b)` — checks whether two qubits share an entanglement group.
- `remove(x)` — used by `remove_qubit`, returns `False` if x is still entangled.

R.8.6 State Aliasing

The method `rename_state(old_id, new_prefix)` creates an alias for an existing simulator state under a new prefix. This is essential when qubits are transferred between places: the received qubit gets a new alias with the receiver's prefix but still references the same simulator state.

R.9 Multiprocessing Model

Each place runs as a separate OS process (`multiprocessing.Process`).

Key consequences:

- Parse trees must be re-created in each process.
- Shared state requires IPC primitives like `multiprocessing.Queue` and `multiprocessing.Manager`.

R.9.1 Packet Network Internals

The `QuantumNetwork` class (`int/network.py`) is backed by:

```
manager      = multiprocessing.Manager()
packet_pool  = manager.list()                    # shared packet list
condition    = manager.Condition(manager.Lock())  # for wait/notify
global_counter = manager.Value('i', 0)           # monotonic packet ID
```

Sending (`send`) acquires the condition lock, appends a `Packet` to `packet_pool`, increments the counter, and calls `condition.notify_all()`.

Receiving (`wait_for`) acquires the condition lock and scans the pool for a matching packet. If no match is found, it calls `condition.wait()` and re-scans when notified. Matching considers: quantum/classical flag, data size, target ID, source ID, and message name.

Peeking (`peek`) performs the same scan but does not consume or block.

R.9.2 Per-Place Console

Each place opens a separate CMD window via the `Console` class (`int/console.py`). The window is a subprocess running `console_worker.py`, connected to the place process through a TCP socket on `localhost`.

R.10 Error Reporting Internals

The `ScriptErrors.showError()` method (`int/script_errors.py`) renders a coloured, box-drawn error display.

R.10.1 Runtime Error Verification

For tests that expect a specific runtime error, the framework parses structured `ERROR_*` entries from the log:

```
[--- TEST ---](D) ERROR_LINE_START[0]=5
[--- TEST ---](D) ERROR_LINE_END[0]=5
[--- TEST ---](D) ERROR_COL_START[0]=10
[--- TEST ---](D) ERROR_COL_END[0]=15
[--- TEST ---](D) ERROR_CODE[0]=DBZ-1
```

The helper `find_error_entry(line_start, line_end, col_start, col_end, code)` matches against these entries, allowing tests to verify both the error code and the exact source location.

R.10.2 Syntax Error Collection Mode

A special mode (`SYNTAX_ERROR_COLLECT`) scans all `.ql` files in `tests/syntax_errors/`, runs the interpreter in `SYNTAX_MODE` (parse-only, no execution), and collects error data into `tests/syntax_errors.json`. This is used to validate that the custom `QLangErrorListener` produces meaningful messages for every syntax error pattern.

R.11 Key Architectural Decisions

Decision	Rationale
Stabiliser tableau instead of state vector	$O(n^2)$ per-gate cost vs. $O(2^n)$ memory for state vectors; sufficient for the supported Clifford gate set.
OS processes instead of threads	Bypasses Python's GIL, giving genuine parallelism for places.
Re-parsing per place	ANTLR 4 CST objects are not picklable; storing text and re-parsing is the simplest workaround.
Handler dictionary instead of ANTLR Visitor	Enables modular, file-per-handler organisation; avoids a monolithic Visitor class.
TCP sockets for console I/O	Connects the place process to a separate CMD window process on Windows, allows independent lifecycle management.

R.12 Limitations

- **No non-Clifford gates.** The stabiliser formalism cannot simulate gates outside the Clifford group (e.g. T, Toffoli, arbitrary rotations). Universal quantum computation is therefore not expressible.
- **Re-parsing overhead.** Each place re-parses its code at start-up, which adds latency proportional to the code size.